Smart Grid Stability Monitoring and Automated Fault Diagnostic Platform Using MATLAB/Simulink and a Python Streamlit Interface

Madhura Dhondu Shinde
Department of Electrical Engineering, Veermata Jijabai Technological Institute (VJTI), Mumbai, India
[email]

Abstract—Modern grids with distributed generation and power-electronic interfaces need protection that detects, characterizes and isolates faults quickly. This paper presents a software-dominant platform that couples a MATLAB/Simulink grid model with a Python Streamlit dashboard through the MATLAB Engine API. The model contains a synchronous machine, a two-winding transformer, a series RLC branch, two PWM-controlled voltage-source converters, a three-phase breaker, a π-section line, a parallel RLC load and a programmable three-phase fault block. A signal-processing chain computes voltage and current magnitude and angle; a fault-detection block, protection logic and trip-signal generator then drive the breaker. The dashboard lets the user change source voltage, power factor, line R-L-C and fault resistance, runs the model, computes peak current and P/Q estimates, classifies fault severity, shows waveforms and raises an audible alarm. Measured results are reported as placeholders to be filled from the user's runs; machine-learning classification and hardware validation are positioned as future work.

Index Terms—Smart grid, fault diagnosis, power-system protection, MATLAB/Simulink, MATLAB Engine API, Streamlit, circuit breaker, voltage-source converter, digital twin.

I. INTRODUCTION

Renewable generation, converter-interfaced sources and variable loads change fault-current levels and voltage profiles, so protection must detect abnormal conditions and also report their type, severity and consequences [1], [2]. Conventional overcurrent and distance schemes rely on fixed thresholds and offer little visualization or event analysis. This work builds an integrated, simulation-driven platform that (i) models the grid and its protection chain in Simulink, (ii) exposes the main electrical parameters through a web dashboard, and (iii) reports fault severity, mitigation advice and an alarm.

Contributions: (1) a Simulink model with converter-interfaced sources, a protection chain and a fault injection block; (2) a Python automation script that runs the model and produces a five-panel publication plot; (3) a Streamlit interface with live parameter editing, metrics, severity-based mitigation guidance, waveform diagnostics and a web-audio alarm.

Condition	Severity
Three-phase fault or I_{pk} > 10 000 A	Critical (symmetrical)
Line-to-line fault or I_{pk} > 5 000 A	High (asymmetrical)
Otherwise	Moderate (ground fault)

TABLE III. Severity rules in app.py.

Each class carries mitigation advice: Zone 1 distance trip with auto-reclose inhibited, STATCOM/FACTS support and surge-arrester checks (critical); delayed Zone 2 trip and ANSI 46 negative-sequence protection (high); single-pole auto-reclosure, grounding-reactor tuning and arc-resistance compensation (moderate). These are advisory text and are not computed by the simulation.

VI. SOFTWARE IMPLEMENTATION

A. run_sim.py

guidance, waveform diagnostics and a web audio alarm.

II. SYSTEM ARCHITECTURE

Fig. 1 shows the data flow: Streamlit sets block parameters via the MATLAB Engine, Simulink simulates, and the logged signals return to Python as NumPy arrays for analysis and display.

```
Streamlit UI -> set_param() -> Simulink main_model
     ^                              |
     |                    To Workspace "ScopeData"
 metrics/plots/alarm <- NumPy <- scopeObj.signals.values
```

Fig. 1. Software architecture (the Simulink screenshot is Fig. 2).

III. SIMULINK MODEL

The model *main_model* uses the Specialized Power Systems blocks with a discrete powergui solver (sample time 5×10^{-5} s), stop time 1 s and a variable-step discrete solver.

A. Power Circuit

Block	Role
Synchronous Machine (pu standard)	Main source; inputs Pm and Vf both set to 1
Three-Phase Parallel RLC Branch1	Grounded resistive branch at the machine terminals (neutral/damping path)
Three-Phase Transformer (two windings)	Step-down, Yg–Yg connection
Three-Phase Series RLC Branch	Source/line impedance; R, L, C set from the dashboard
Three-Phase V-I Measurement	Outputs Vabc and Iabc to the processing chain
Series VSC 1 / Series VSC 2	Converter stages, gated by PWM Generator and PWM Generator1 (1×6 pulse constants); a capacitor bank with grounded midpoint sits on the DC link
Three-Phase Breaker	Controlled by Trip_CB
Three-Phase PI Section Line	Transmission line

Starts the MATLAB Engine, loads *main_model*, runs *sim*, reads *ScopeData*, transposes the array if its orientation differs from the time vector, and plots five panels (Vabc, Iabc, trip status, Vmag, Imag), saved as *simulation_results.png* at 300 dpi.

B. app.py (Streamlit)

The engine is cached with *st.cache_resource* and the model is loaded once per session. Sidebar inputs:

Parameter	Range	Default
Primary voltage	1–132 kV	11 kV
Secondary voltage	100–11 000 V	415 V
Power factor	0.50–1.00	0.85
Line R	0.01–100 Ω	1 Ω
Line L	0.1–1000 mH	10 mH
Line C	0.01–100 µF	1 µF
Fault resistance R_f	0.001–50 Ω	5 Ω
Fault type	AG, AB, ABC	AG

TABLE IV. Dashboard parameters.

On *Run*, parameters are written with *set_param* (source amplitude, series RLC, fault resistance), the model is simulated, data are extracted, and the page shows metrics (step-down ratio, power factor, peak current, P, Q), the severity and mitigation panel, and three stacked waveform plots. If the trip signal drops below 0.5, an error banner is shown and a 1 kHz sawtooth tone plays for 3 s via the Web Audio API.

VII. RESULTS AND ANALYSIS

Figures to include: Fig. 2, the Simulink screenshot; Fig. 3, the *simulation_results.png* panels showing the 0.2–0.6 s fault period. Insert your measured values below; none are claimed here.

Case	R_f	I_{pk} (A)	V_{min} (V)	Trip time (ms)	Severity
Normal	–	XX	XX	–	–
AG	5 Ω	XX	XX	XX	XX
AB	5 Ω	XX	XX	XX	XX

Three-Phase Parallel RLC Load	Load at the receiving end
Three-Phase Fault	Fault injection; fault resistance set from the dashboard

TABLE I. Power-circuit components in main_model.

B. Protection and Logging Chain

Signal Processing converts Vabc and Iabc into Vmag, Vang, Imag and Iang. Fault Detection takes Vmag and Imag and outputs a Fault flag. Protection Logic converts the flag to Trip. The Trip Signal Generator produces Trip_CB for the breaker, with Trip passed through a Data Type Conversion (double). A Bus Creator then packs nine signals into *ScopeData* (To Workspace, structure format) and also feeds a Scope.

Col.	**Signal**	**Col.**	**Signal**
0–2	Va, Vb, Vc	6	Trip (0 = open)
3–5	Ia, Ib, Ic	7	Vmag
8	Imag		

TABLE II. ScopeData column map.

IV. MATHEMATICAL FORMULATION

RMS value over N samples:

$$X_{RMS} = \sqrt{((1/N) \Sigma x_n^2)}$$

Phasor magnitude and angle (Signal Processing block) give Vmag, Vang, Imag, Iang. The dashboard estimates power from the simulated peak current I_{pk} and the user-selected power factor cos φ:

$$S = \sqrt{3} \cdot V_{LL} \cdot I_{pk}/1000 \text{ (kVA)}, \; P = S \cos \varphi, \; Q = S \sin(\arccos \varphi)$$

The programmable source phase peak is set from the line voltage as

$$V_{peak,ph} = V_{LL} \cdot \sqrt{2}/\sqrt{3}$$

For symmetrical-component analysis (planned extension) with $a = e^{j120°}$:

$$[V_0 \; V_1 \; V_2]^T = (1/3)[[1,1,1],[1,a,a^2],[1,a^2,a]][V_a \; V_b \; V_c]^T$$

Per-unit bases: $Z_{base} = V_{base}^2/S_{base}$, $I_{base} = S_{base}/(\sqrt{3}\ V_{base})$. A simplified

ABC	5 Ω	XX	XX	XX	XX

TABLE V. Measured results (to be completed). Trip time = t(Trip falls) − t(fault inception).

Also report sensitivity to R_f (0.001–50 Ω), line L and C, and source voltage, using the same metrics.

VIII. DISCUSSION AND LIMITATIONS

1) The fault-type selector in the dashboard changes only the reported severity text; the code does not set the Three-Phase Fault block's phase-selection parameters, so the simulated fault is whatever the model is configured for. Add *set_param* calls for the A/B/C/Ground fault flags before comparing fault types. **2)** Secondary voltage and the power-factor slider do not affect the simulation; P and Q are derived from the peak current. **3)** Real/reactive power is better computed from instantaneous v·i in the model. **4)** The severity logic is rule-based; the "AI" label in the interface should be described as rule-based. **5)** The 15 ms and 300 ms figures are advisory settings, not measurements. **6)** The two bare *except* clauses on the source update can hide a wrong parameter name. **7)** The fault window (0.2–0.6 s) is hard-coded and should be read from the fault block. **8)** Results come from simulation only; a trained classifier or hardware would need independent validation.

IX. FUTURE WORK

Random Forest classification of ten fault classes using RMS, sequence components and THD; impedance-based location; MATPOWER power-flow coupling; Arduino low-voltage relay demonstration with serial telemetry; PMU and COMTRADE ingestion; event logging with PDF reports; deep-learning classifiers.

X. CONCLUSION

The platform integrates a Simulink grid with converter-interfaced sources, a detect-trip-isolate chain and a Streamlit interface for parameter studies, severity analysis, waveform diagnostics and alarms. Completing the measurement tables and correcting the limitations above will give a credible basis for the intelligent diagnosis and hardware stages.

impedance-based location estimate is $d = (|V/I|/|Z_{line}|)\cdot L$, valid only as a first approximation (fault resistance, infeed and mutual coupling are ignored).

V. PROTECTION ALGORITHM

The detection uses Vmag and Imag. A fault is declared when voltage sags or current exceeds its limit, the trip output goes low (breaker open), and the dashboard declares a fault if any sample of the trip signal is below 0.5. The fault window of 0.2–0.6 s is shaded on all plots. The dashboard then classifies severity from the selected fault type or peak current:

REFERENCES

[1] P. Kundur, *Power System Stability and Control*. New York, NY, USA: McGraw-Hill, 1994.

[2] P. M. Anderson, *Power System Protection*. New York, NY, USA: IEEE Press/McGraw-Hill, 1999.

[3] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Trans. Power Syst.*, vol. 26, no. 1, pp. 12–19, Feb. 2011.

[4] *IEEE Recommended Practice for Monitoring Electric Power Quality*, IEEE Std 1159-2019.

[5] *IEEE Guide for Protective Relay Applications to Transmission Lines*, IEEE Std C37.113.

[6] MathWorks, "MATLAB Engine API for Python" and "Simscape Electrical documentation."